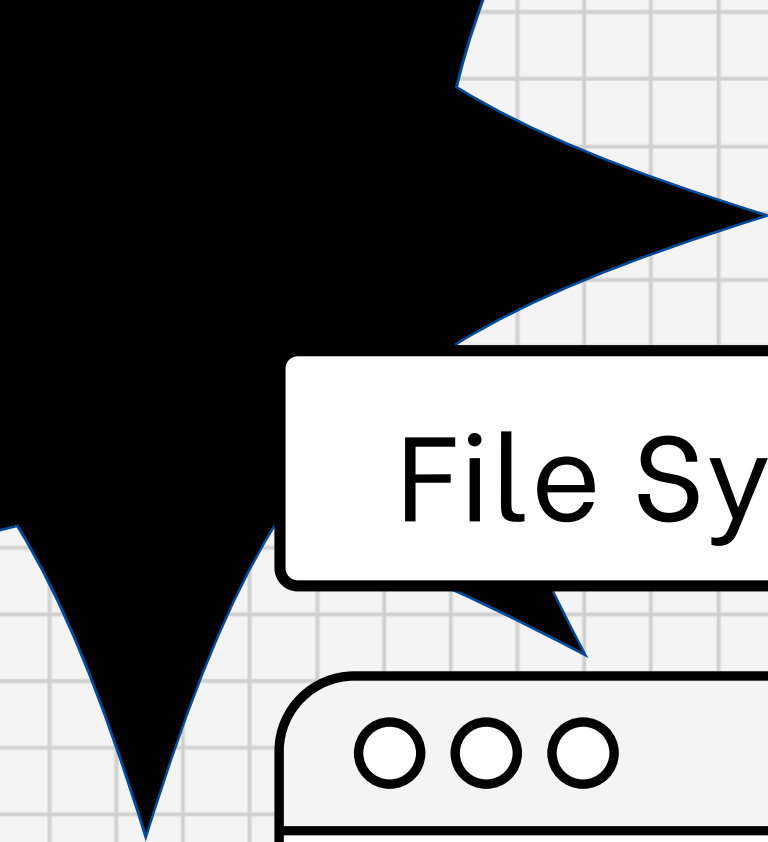




File System

Case_3



Построение эмпирической модели производительности in-memory файловой системы

ITMO-Avito-Team7

Задачи работы

- + Реализация трёх различных подходов к in-memory файловой системе.
- + Разработка параметризуемого бенчмарка, способного измерять производительность в любой точке пространства параметров.
- + Проведение систематических измерений в узлах решётки параметров.
- + Построение аппроксимирующих функций для каждого из трёх подходов, предсказывающих метрики качества.
- + Реализация программы-рекомендателя, выбирающей оптимальный подход для заданных условий на основе предсказаний моделей.

Реализация трёх различных подходов к in-memory файловой системе

Операция	Подход А: N-арное дерево
read(path)	$O(D * W)$
write(path) / перезапись	$O(D * W)$
create file	$O(D * W)$
mkdir(path)	$O(D * W)$
ls(path)	$O(D * W + K)$
mv(path)	$O(D * W + W)$
find(path, pattern)	$O(D * W + M)$

- сильно зависит от глубины, потому что путь ищется последовательным проходом по дереву. На каждом уровне нужно найти нужного ребёнка среди списка детей
- сильно замедляется при росте ширины дерева, потому что каждый узел хранит список детей, и поиск нужного ребёнка может требовать просмотра большого списка
- изменение доли файлов влияет на форму дерева и количество элементов в списках детей

N-арное
дерево

Реализация трёх различных подходов к in-memory файловой системе

Операция	Подход В: дерево + hash map путей
read(path)	$O(1)$ average, фактически $O(L)$
write(path) / перезапись	$O(1)$ average
create file	$O(1)$ average
mkdir(path)	$O(1)$ average
ls(path)	$O(1 + K)$
mv(path)	$O(W + S * L)$
find(path, pattern)	$O(1 + M)$

- ▶ в меньше зависит от глубины пути, полный путь можно найти через hash map
- ▶ меньше зависит от среднего числа детей в директории при поиске по полному пути, тк использует hash map. однако ls зависит от количества детей в директории
- ▶ рост числа файлов увеличивает размер дополнительной hash map, но сохраняет быстрый доступ по пути

Дерево

Хэш-таблица
полных путей

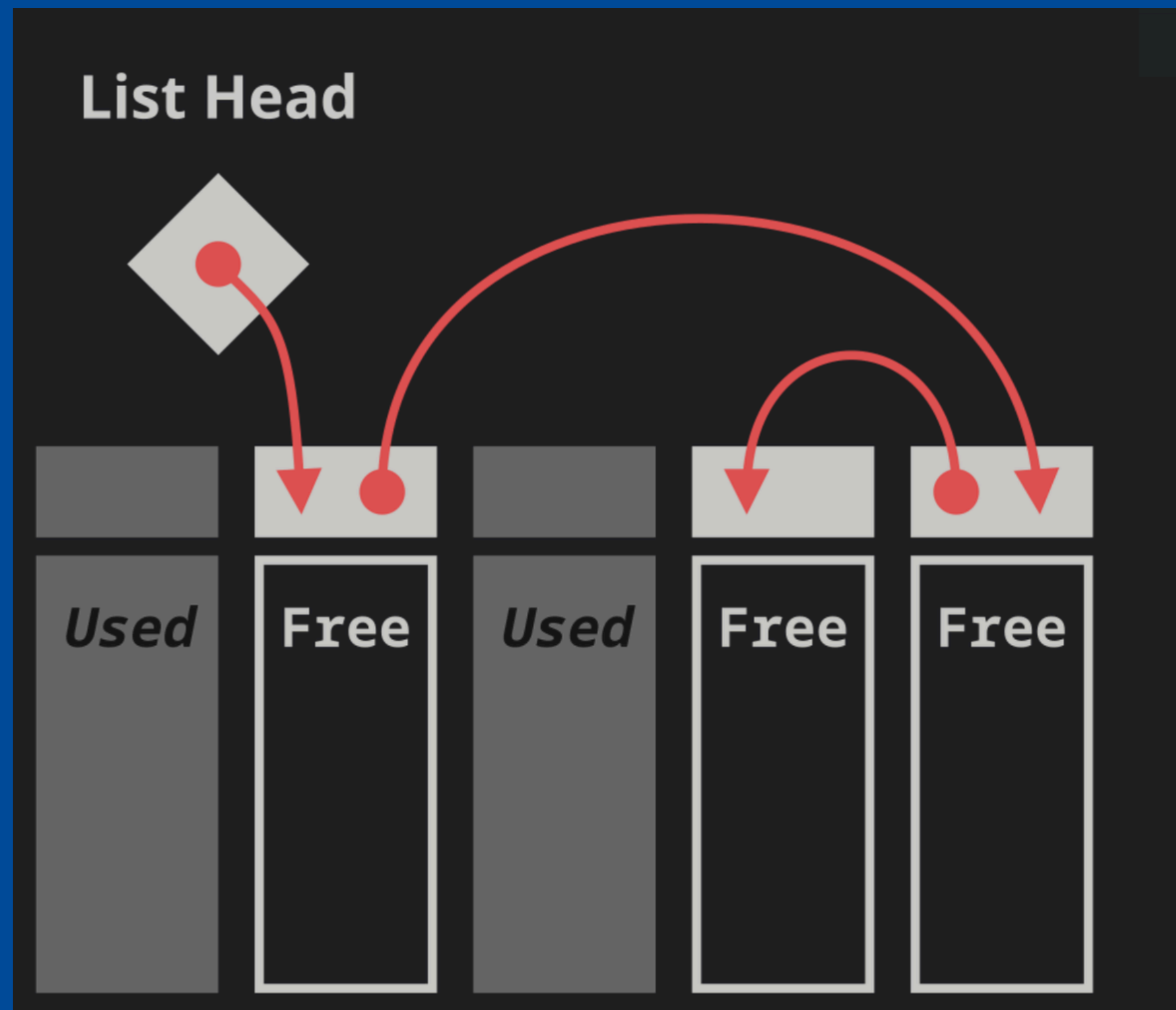
Реализация трёх различных подходов к in-memory файловой системе

Операция	Подход C: плоская hash map
read(path)	$O(1)$ average, фактически $O(L)$
write(path) / перезапись	$O(1)$ average
create file	$O(1)$ average
mkdir(path)	$O(1)$ average
ls(path)	$O(N)$
mv(path)	$O(N * L)$
find(path, pattern)	$O(N)$

- тоже меньше зависит от глубины пути для точного доступа, потому что путь является ключом в hash map. длинные пути увеличивают стоимость хэширования и сравнения строк
- почти не зависит от среднего числа детей в директории при read и write, для ls ему может быть тяжело, поскольку без дерева нужно искать элементы с нужным префиксом среди всех путей
- большое число файлов увеличивает кол-во записей в плоской таблице, из-за чего операции по точному пути остаются быстрыми, но операции, требующие анализа иерархии, могут становиться медленнее

Плоское хэш-отображение

Free-List аллокатор



benchmark

автоматическое измерение производительности трёх реализаций
in-memory файловой системы на различных конфигурациях нагрузки

1

- построение начального состояния ФС
- генерация последовательности операций
- запуск идентичной нагрузки на подходах A, B и C
- измерение latency, p99, throughput и памяти
- сохранение результатов в CSV для последующего обучения модели

2

Архитектура

Компонент	Назначение
WorkloadConfig	Хранит параметры одной конфигурации нагрузки
WorkloadGenerator	Генерирует setup-операции и measured-операции
BenchmarkRunner	Выполняет операции и измеряет метрики
CsvWriter	Записывает результаты в CSV
main.cpp	Обходит сетку параметров и запускает все измерения

3

Генерация нагрузки

- Setup - фаза
- Measured-фаза

benchmark

автоматическое измерение производительности трёх реализаций in-memory файловой системы на различных конфигурациях нагрузки

- 864 конфигурации нагрузки
- Каждая конфигурация запускается на трёх подходах - в CSV сохраняется 2592 строки
- При Ops = 5000 и Repeats = 7 выполняется: $864 \cdot 3 \cdot 7 = 18144$ изм. запусков
- Общее количество измеряемых операций: 90_720_000

4

6 прикладных профилей

Профиль	read	write	mkdir	ls	mv	find
build_system	0.45	0.30	0.10	0.05	0.05	0.05
file_manager	0.20	0.10	0.15	0.30	0.20	0.05
repo_server	0.60	0.20	0.05	0.05	0.03	0.07
backup	0.55	0.05	0.05	0.20	0.02	0.13
refactoring	0.20	0.15	0.10	0.10	0.35	0.10
static_web	0.85	0.03	0.01	0.03	0.01	0.07

5

Измеряемые метрики

Метрика	Описание
avg_latency_us	Средняя задержка операции в микросекундах
p99_latency_us	99-й процентиль задержки
throughput_ops_sec	Количество операций в секунду
memory_bytes	Оценка потребления памяти

6

Формат CSV

- название подхода;
- параметры D, W, F;
- профиль операций;
- вероятности p_read, p_write, p_mkdir, p_ls, p_mv, p_find;
- параметры распределения dist, zipf_s, locality;
- ops и repeats;
- средние значения метрик;
- стандартные отклонения метрик.

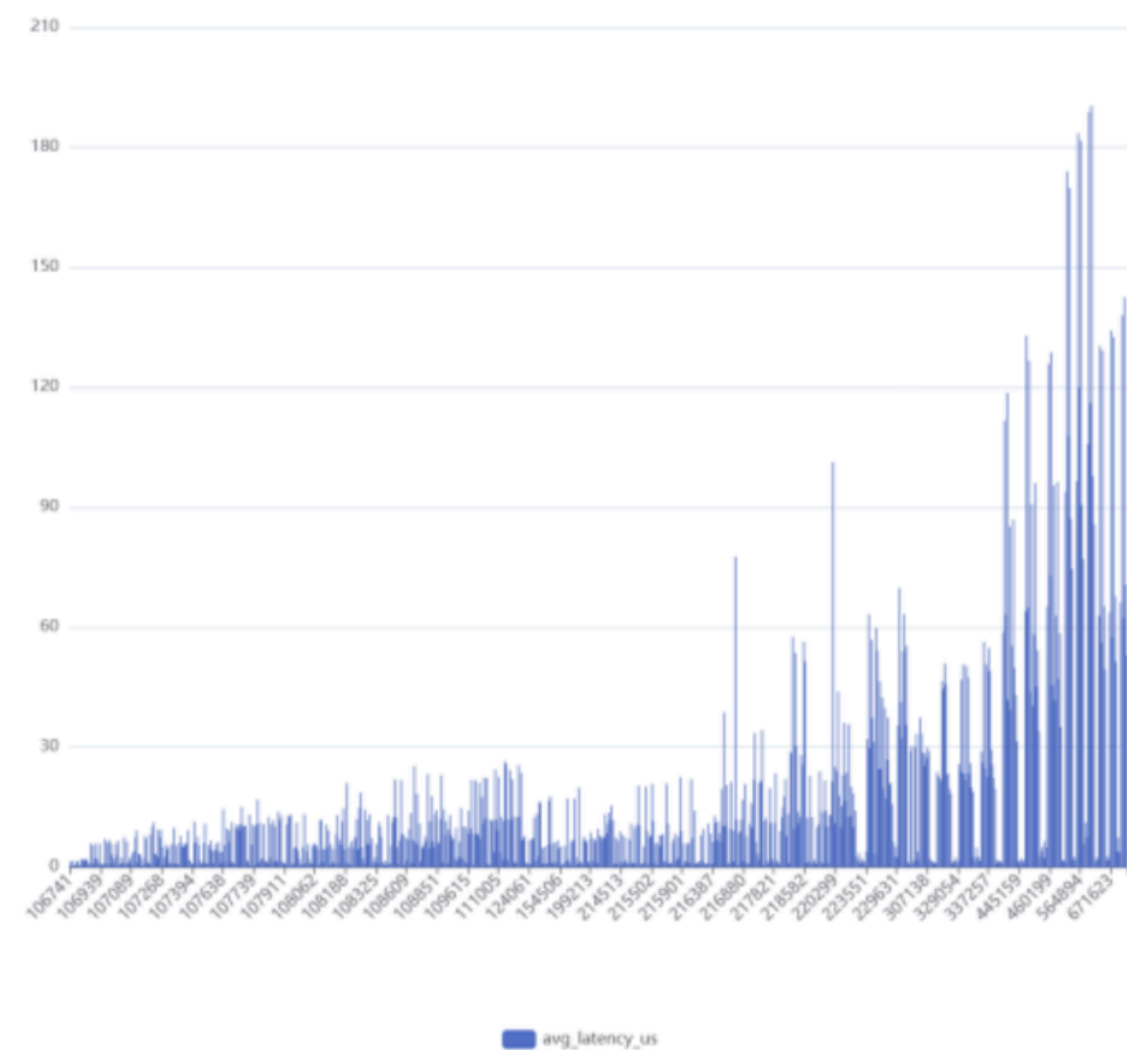


Разработанный benchmark:

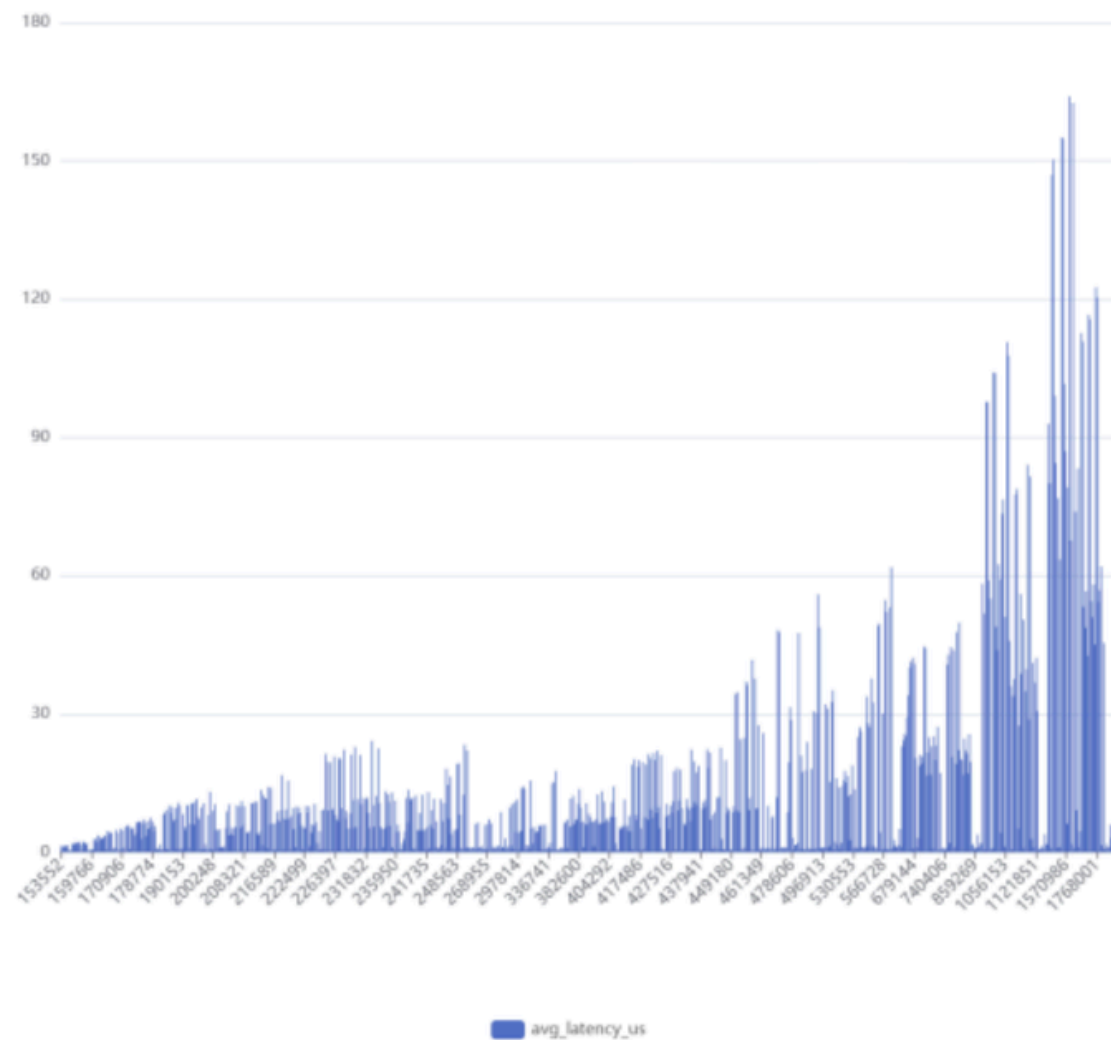
- запускает три подхода A, B и C;
- использует параметризуемую сетку;
- поддерживает прикладные профили операций;
- учитывает распределение доступа и локальность;
- измеряет latency, p99, throughput и memory;
- агрегирует результаты по нескольким повторам;
- сохраняет CSV для анализа и обучения модели.



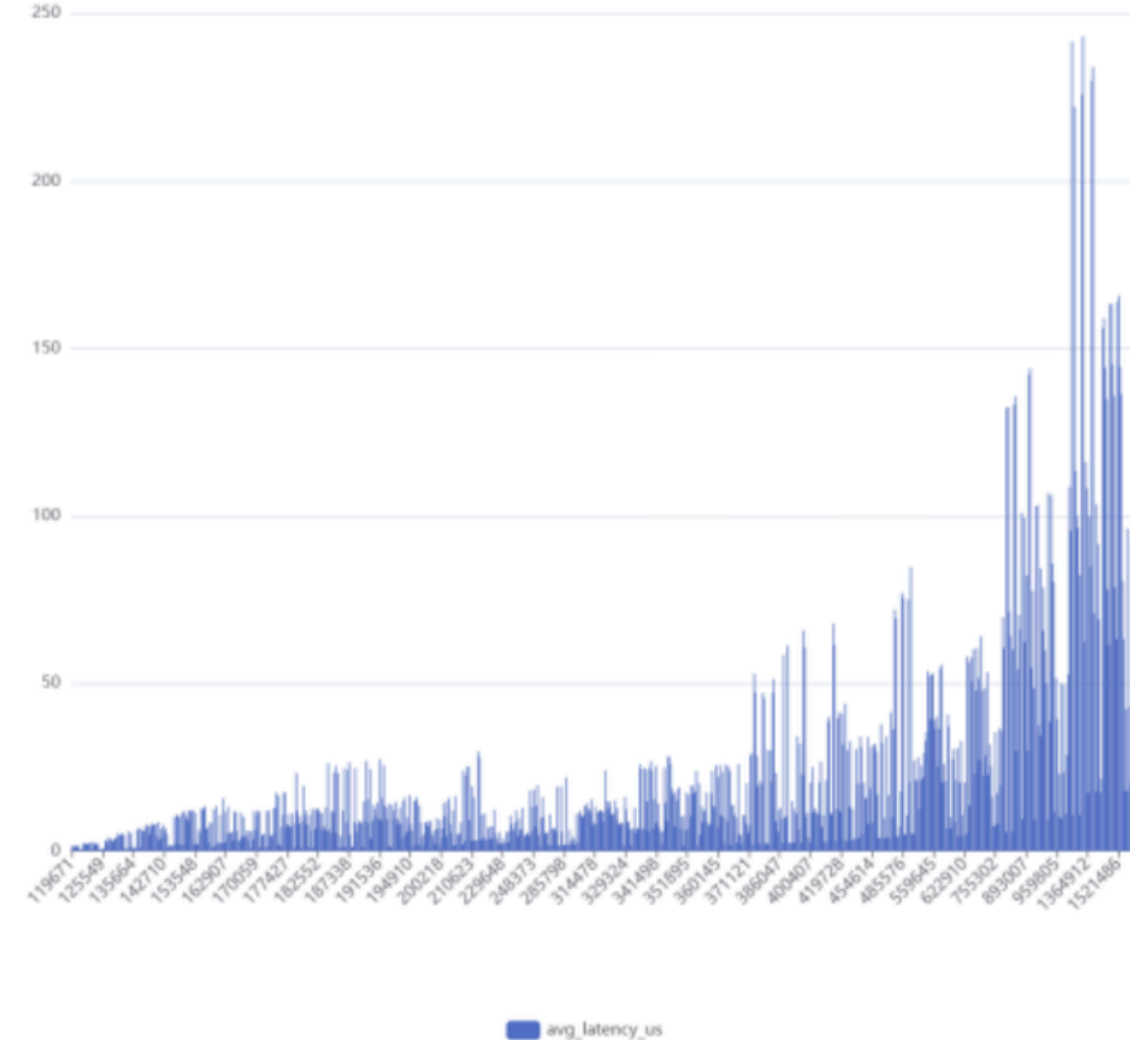
avg_latency_us vs memory_bytes



A



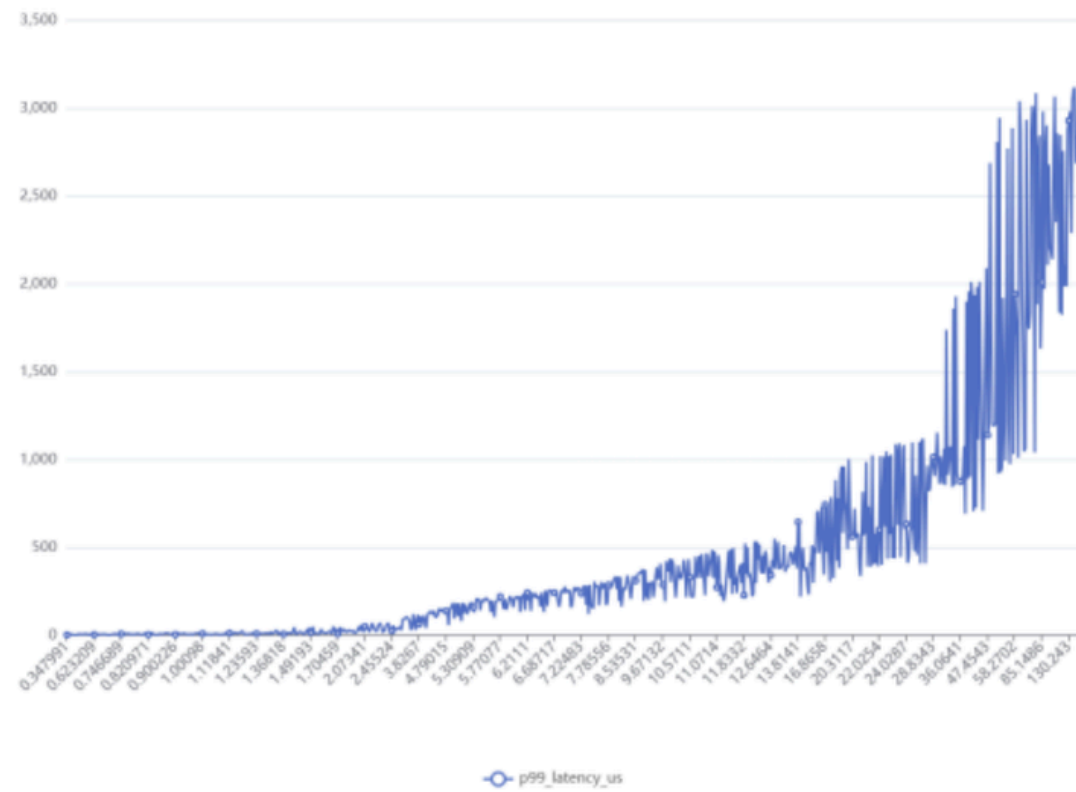
B



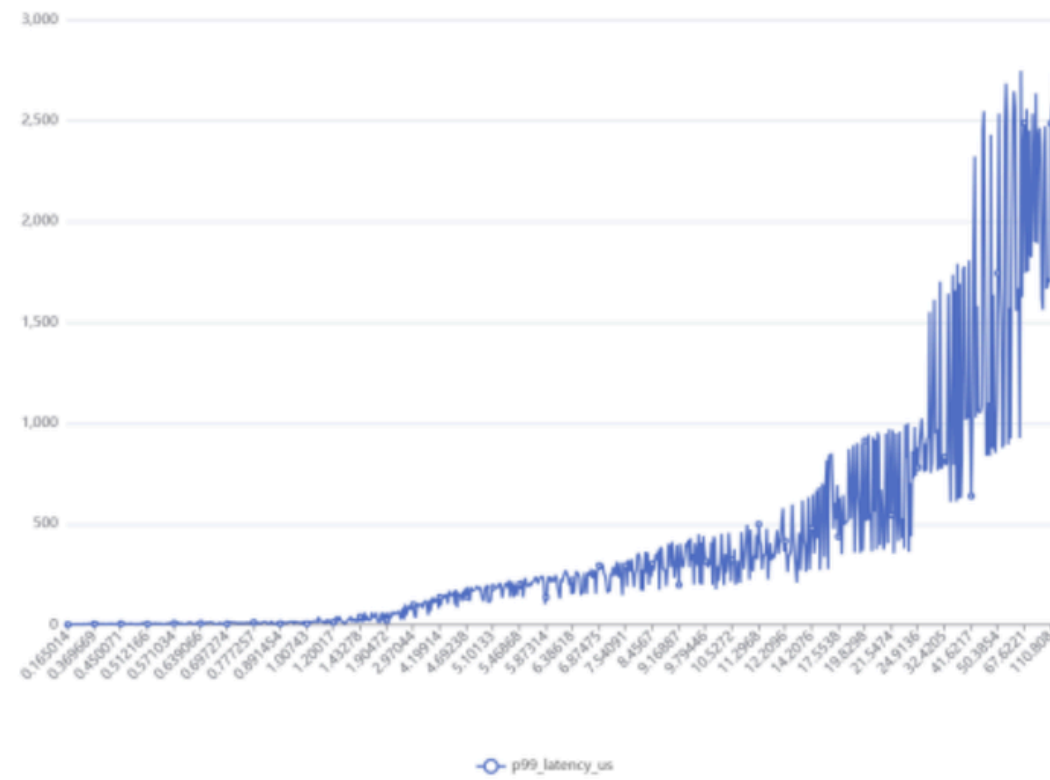
C

Зависимость средней латентности от потребления памяти для подходов A, B и C

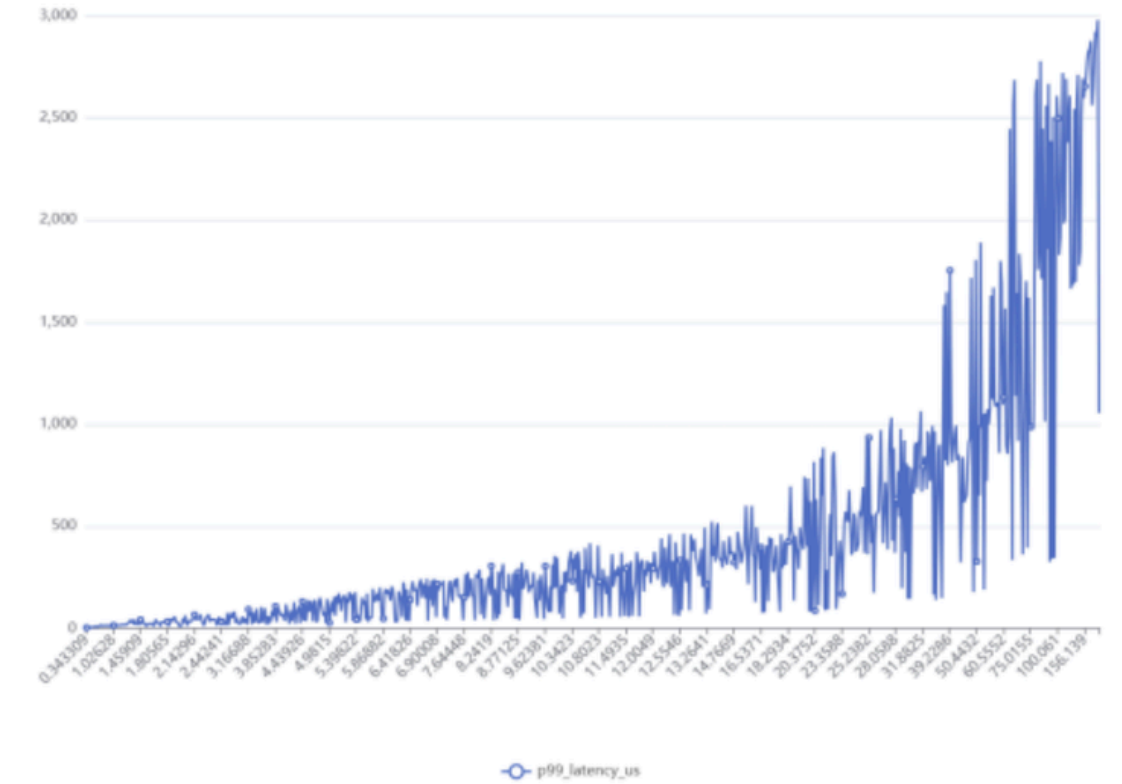
avg_latency_us vs p99_latency_us



A



B



C

Связь средней латентности и p99 latency для подходов A, B и C

ML-Model

предсказание производительности реализаций файловой системы: A, B и C

1

Предсказание 4 метрик:

- `avg_latency_us` – средняя задержка операции в микросекундах;
- `p99_latency_us` – 99-й процентиль задержки в микросекундах;
- `memory_bytes` – потребление памяти в байтах;
- `throughput_ops_sec` – пропускная способность в операциях в секунду.

2

Проблема комбинаторного взрыва и выбор аппроксимирующей функции

Критерий	Random Forest
Форма зависимостей	Нелинейная, с сглаживанием порогов
Объём данных	Средний
Интерпретируемость	Низкая
Вычислительные ресурсы	Низкие-умеренные
Корреляции между метриками	Устойчива

Random Forest — ансамблевый алгоритм машинного обучения, основанный на построении множества деревьев решений. Ключевая идея — комбинация нескольких простых моделей (деревьев) обычно даёт более точный результат, чем одна сложная модель

ML-Model

предсказание производительности реализаций файловой системы: А, В и С

3

Для каждой целевой метрики обучается отдельная модель:

- модель средней задержки
- модель р99 задержки
- модель потребления памяти
- модель пропускной способности

4

Последовательность работы:

1. Загружается results.csv.
2. Категориальные значения approach и dist преобразуются в числа.
3. Данные делятся на обучающую и тестовую части.
4. Из best_params.json загружаются лучшие параметры.
5. Для каждой из четырех метрик создается отдельная модель.
6. Модели обучаются на обучающей выборке.

На тестовой выборке создается таблица предсказаний ***y_pred***.

Вычисляются общие ***R2***, ***MAE*** и ***RMSE***.

Модели и служебная информация сохраняются в model.pkl.

ML-Model

предсказание производительности реализаций файловой системы: A, B и C

5

Пользовательский интерфейс

Принимает:

- D, W, F – параметры структуры;
- p_read, p_write, p_mkdir, p_ls, p_mv, p_find – вероятности операций;
- dist – uniform или zipf;
- zipf_s – параметр распределения Ципфа;
- locality – локальность обращений;
- strategy – стратегия выбора подхода.

6

Для каждого входного набора функция формирует три строки признаков:

1. approach = 0 для A
2. approach = 1 для B
3. approach = 2 для C

Поддерживаются пять стратегий.

- latency
- p99
- memory
- throughput
- balanced

7

Комбинированная стратегия учитывает все четыре метрики с весами

- avg_latency_us 0.4
- p99_latency_us 0.2
- memory_bytes 0.2
- throughput_ops_sec 0.2

ML-Model

Пара метрик	Pearson	Spearman	Вывод
avg latency и p99	0.0639	0.9517	Сильная монотонная, но не линейная зависимость
avg latency и throughput	-0.0305	-0.9998	Почти строгая обратная зависимость
p99 и throughput	-0.3875	-0.9511	При росте p99 throughput обычно падает
p99 и memory	0.5917	0.3725	Умеренная связь
memory и throughput	-0.1329	-0.4011	Слабая или умеренная обратная связь

Вывод: p99 не является линейной функцией от средней латентности. Между ними есть сильная монотонная зависимость, но она нелинейная и чувствительная к редким медленным операциям. Поэтому p99 нужно предсказывать отдельной моделью, а не вычислять напрямую из avg_latency_us.

ML-Model

Подход	Метрика	R ²	MAE	RMSE
A	avg latency	0.9918	1.1832	2.6382
A	p99 latency	0.9946	20.8384	49.4080
A	memory	0.9997	827.8958	2393.6639
A	throughput	0.9862	27917.4287	58124.4792
B	avg latency	0.9612	1.8994	4.7076
B	p99 latency	0.9980	13.7487	24.1759
B	memory	0.9996	4645.2497	7155.9053
B	throughput	0.9759	50089.4447	122349.0780
C	avg latency	0.9108	4.0615	10.6542
C	p99 latency	0.9950	14.3267	42.3931
C	memory	0.9996	4343.2979	6409.3296
C	throughput	0.9610	20967.0900	68185.6874

Качество было рассчитано на той же тестовой выборке 20%, которая формируется в `ModelLearner.py` с `random\_state=42`. Для каждого подхода и каждой метрики рассчитаны R², MAE и RMSE.

Принятие бизнес-решения



Параметры нагрузки и стратегия на входе	<code>recommend.py (argparse)</code>
Метрики A/B/C + стратегия + рекомендация на выходе	<code>format_report()</code> , флаг <code>--json</code>
Без запуска бенчмарка, только арифметика	<code>predict.py + strategies.py</code>
Несколько стратегий, в т.ч. с ограничениями	<code>strategies.py</code> : 9 стратегий
Пайплайн: сборка → прогон → CSV → обучение	<code>pipeline.py</code>
Демонстрация на примерах	раздел «Демонстрация»
Смена стратегии меняет выбор подхода	таблица стратегий в демо

**ML предсказывает метрики →
рекомендатель по выбранной стратегии выбирает подход → пайплайн позволяет
воспроизвести данные и модель офлайн**

Стратегии выбора

оптимального
подхода “в вакууме” не существует, система поддерживает три класса стратегий,
отражающих разные инженерные приоритеты

1

Однометричные стратегии (latency, p99, memory, throughput)

Применяются, когда в системе есть жестко доминирующий приоритет.

Выбирается один критерий, и по нему определяется абсолютный победитель (например, минимальное потребление RAM для embedded-устройств)

2

Взвешенные стратегии (balanced, *-critical)

Используются, когда необходимо найти компромисс между метриками. Поскольку метрики имеют разную размерность (микросекунды, байты, операции/сек), применяется метод Min-Max нормализации штрафов (от 0 до 1) исключительно по трем рассматриваемым подходам (A, B, C). Итоговый выбор делается на основе взвешенной суммы штрафов. Это позволяет избежать ситуации, когда подход выигрывает миллисекунду скорости, но требует в 10 раз больше памяти.

3

Стратегии с ограничениями (constrained)

1. Отсекаются подходы, не проходящие заданные лимиты (например, `p99_latency_us <= 5000` мкс, то есть 5 мс).
2. Среди оставшихся выбирается лучший по целевой метрике (--objective). Если ограничения невыполнимы ни для одного подхода, система не падает с ошибкой, а вычисляет суммарное относительное нарушение и предлагает “наименьшее из зол”, явно предупреждая пользователя.

Dev 4 в Interface.py поддерживает 5 стратегий выбора. Dev 5 расширяет набор до 9: добавлены профили *-critical и стратегия constrained.

Класс	Стратегия	Критерий / суть
Однометричные (4)	latency	min avg_latency_us
	p99	min p99_latency_us
	memory	min memory_bytes
	throughput	max throughput_ops_sec
Взвешенные (4)	balanced	веса 0.4 / 0.2 / 0.2 / 0.2
	latency-critical	латентность доминирует (0.5 / 0.3 / 0.1 / 0.1)
	memory-critical	память доминирует (0.6 / 0.2 / 0.1 / 0.1)
	throughput-critical	throughput доминирует (0.6 / 0.2 / 0.1 / 0.1)
С ограничениями (1)	constrained	фильтр по SLA + --objective

Таблица весов взвешенных профилей

Профиль	avg lat.	p99	memory	throughput
balanced	0.40	0.20	0.20	0.20
latency-critical	0.50	0.30	0.10	0.10
memory-critical	0.20	0.10	0.60	0.10
throughput-critical	0.20	0.10	0.10	0.60

Параметры CLI

Группа	Параметры	Смысл
Структура	--D, --W, --F	глубина, ширина, доля файлов
Операции	--p-read ... --p-find	вероятности типов операций
Доступ	--dist, --zipf-s, --locality	uniform/zipf, локальность
Стратегия	--strategy	одна из 9 стратегий
Ограничения	--max-p99, --max-memory, ...	только для constrained
Вывод	--json	машиночитаемый JSON

Демонстрация работы и ключевой вывод

Базовая нагрузка: $D = 10$, $W = 100$, $F = 0.6$; профиль операций близок к hero_server ($p_{\text{read}}=0.6$, $p_{\text{write}}=0.2$, ...); dist=zipf, zipf_s=1.5, locality=0.3.

Предсказанный вектор метрик (ML-модель Dev 4):

Подход	avg lat., мкс	p99, мкс	memory, байт	throughput, оп/с
A	21.8	765.9	220833	43834
B	21.0	679.2	480973	47859
C	24.0	682.8	400902	42062

Как меняется рекомендация при той же нагрузке:

Стратегия	Выбор	Почему
memory	A	минимальная память (220 KB vs 481 KB у B)
latency	B	минимальная avg latency (21.0 мкс)
p99	B	минимальный p99 (679.2 мкс)

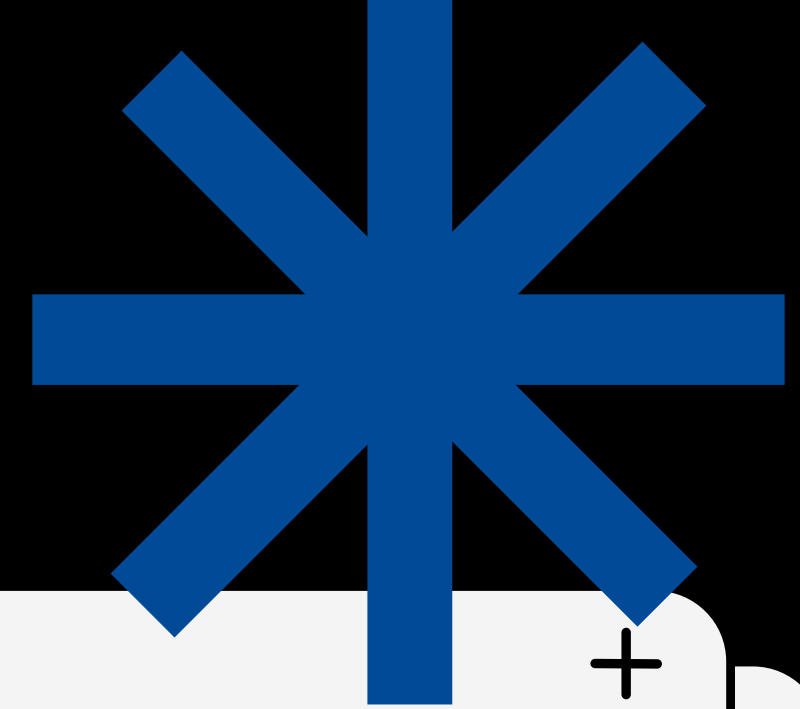
throughput	B	максимальный throughput
balanced	B	компромисс: B лучше по latency/p99/throughput
latency-critical	B	доминирует латентность
constrained (memory, p99 $\leq$ 5000 мкс)	A	все проходят p99; min memory = A

1. Пользователь передаёт параметры и стратегию в `recommend.py`.
2. `predict.py` вызывает `getPredictions` Dev 4 и извлекает `predictions` по A/B/C.
3. `strategies.py` применяет стратегию и возвращает `Recommendation`.
4. CLI форматирует текстовый отчёт или JSON.

Формально: `params` $\rightarrow$ `predictions_{A,B,C}` $\rightarrow$ `recommended_approach`.

Структура каталога `recommender/`

Файл	Назначение
<code>strategies.py</code>	9 стратегий выбора поверх <code>predictions</code>
<code>predict.py</code>	обёртка над <code>Interface.getPredictions</code>
<code>recommend.py</code>	CLI и форматированный вывод (§6)
<code>pipeline.py</code>	пайплайн §8.1: <code>smake</code> $\rightarrow$ <code>benchmark</code> $\rightarrow$ <code>CSV</code> $\rightarrow$ <code>train</code>
<code>requirements.txt</code>	Python-зависимости
<code>README.md</code>	документация по запуску



**Спасибо за
внимание**

